

Private Interface Methods

i metodi privati nelle interfacce

Definizione e contesto

A partire da Java 9, le interfacce possono avere metodi privati. Prima di Java 9 questo non era possibile.

Caratteristiche fondamentali di questi metodi:

- Possono essere sia **statici** che **non statici** (di istanza).
- Essendo **privati**, sono accessibili **solo all'interno dell'interfaccia** stessa: chi implementa o usa l'interfaccia dall'esterno non li vede.
- Come nelle classi, vale la regola generale: **non puoi chiamare un metodo non statico da un contesto statico** (un metodo statico non può chiamare direttamente un metodo di istanza).

Perché sono stati introdotti? I due vantaggi

1. **Ridurre la duplicazione del codice** — se più metodi dell'interfaccia condividono la stessa logica, quella logica si può estrarre in un unico metodo privato invece di ripeterla.
2. **Migliorare l'incapsulamento** — nascondere i dettagli dell'implementazione all'esterno dell'interfaccia.

La lezione dimostra entrambi i vantaggi con un esempio concreto: un'interfaccia sul tennis.

Ridurre la duplicazione del codice

La versione INEFFICIENTE (con duplicazione)

Immagina un'interfaccia `TennisInefficiente` con tre metodi — **dritto** (forehand), **rovescio** (backhand) e **smash**. Uno è statico, gli altri due sono default. Ognuno esegue **tre passaggi**:

1. muoversi in posizione (*move into position*)
2. colpire il colpo specifico (dritto / rovescio / smash)
3. tornare in posizione pronta (*return to ready position*)

```
interface TennisInefficiente {
```

```
    static void forehand() {  
        System.out.println("muoversi in posizione"); // duplicato  
        System.out.println("colpisci un dritto");  
        System.out.println("torna in posizione pronta"); // duplicato  
    }
```

```

default void backhand() {
    System.out.println("muoversi in posizione"); // duplicato
    System.out.println("colpisci un rovescio");
    System.out.println("torna in posizione pronta"); // duplicato
}

default void smash() {
    System.out.println("muoversi in posizione"); // duplicato
    System.out.println("colpisci uno smash");
    System.out.println("torna in posizione pronta"); // duplicato
}
}

```

Il problema salta all'occhio: la prima e la terza riga (muoversi in posizione, tornare pronti) sono **identiche in tutti e tre i metodi**. Solo la riga centrale cambia. C'è molta duplicazione.

La versione EFFICIENTE (con metodo privato)

Con Java 9 possiamo estrarre la parte comune in un **metodo privato** che riceve una stringa (il colpo da eseguire):

```

interface Tennis {

    // metodo statico privato: contiene la logica comune
    private static void hit(String stroke) {
        System.out.println("muoversi in posizione"); // parte comune
        System.out.println("colpisci un " + stroke); // parte variabile
        System.out.println("torna in posizione pronta"); // parte comune
    }

    static void forehand() {
        hit("dritto"); // riga 33: chiama il metodo privato
    }

    default void backhand() {

```

```

    hit("rovescio"); // riga 34: chiama il metodo privato
}

default void smash() {
    hit("smash"); // riga 35: chiama il metodo privato
}
}

```

Cosa è successo:

- Il metodo privato `hit` raccoglie la logica comune (muoversi in posizione + tornare pronti), lasciando come parametro solo la parte che varia.
- I tre metodi pubblici (`forehand` statico, `backhand` e `smash` default) ora **chiamano il metodo privato** invece di ripetere il codice.
- **Siamo all'interno dell'interfaccia**, quindi possiamo accedere al metodo privato senza problemi.

Sul fronte delle chiamate, la lezione sottolinea che il metodo statico privato `hit` può essere chiamato senza problemi:

- da un **contesto statico** (il metodo `forehand`) — nessun problema
- da un **metodo default** (`backhand`) — nessun problema
- da un **altro metodo privato** — nessun problema

Risultato: molto meno codice, duplicazione eliminata. **Questo dimostra il primo vantaggio.**

Migliorare l'incapsulamento

Vediamo cosa succede nel `main` quando proviamo a usare l'interfaccia dall'esterno.

```
Tennis tennis = new TennisProfessionale(); // riferimento di tipo Tennis
```

```
tennis.hit("dritto"); // ✗ ERRORE di compilazione
```

Il metodo `hit` **non è accessibile** dall'esterno: è privato, quindi è **incapsulato all'interno dell'interfaccia**. Il compilatore dà errore dicendo che non esiste un metodo `hit` accessibile. Questo è esattamente lo scopo dell'incapsulamento: **nascondere i dettagli dell'implementazione**.

Cosa invece è accessibile dall'esterno:

```
tennis.backhand(); // ✓ metodo default → accessibile tramite l'istanza
```

```
Tennis.forehand(); // ✓ metodo statico → accessibile tramite il nome dell'interfaccia
```

Nota la differenza:

- `backhand` è un metodo **default** (di istanza), quindi lo chiami sull'**oggetto** (`tennis.backhand()`).
- `forehand` è un metodo **statico**, quindi lo chiami sul **nome dell'interfaccia** (`Tennis.forehand()`).

Eseguendo `backhand` e poi `forehand`, l'output sarebbe:

```
muoversi in posizione
colpisci un rovescio
torna in posizione pronta
muoversi in posizione
colpisci un dritto
torna in posizione pronta
```

Questo dimostra il secondo vantaggio.

Le regole sui metodi privati d'interfaccia

Dalla lezione emergono diverse regole importanti.

Regola 1 — Un metodo statico non può chiamare un metodo non statico

Se rendi `hit` **non statico** (togli `static`) ma lo chiami da un metodo statico come `forehand`, ottieni un errore.

```
interface Tennis {
    private void hit(String stroke) { ... } // ora NON statico

    static void forehand() {
        hit("dritto"); // ✗ERRORE: chiami un metodo di istanza da un contesto statico
    }
}
```

Il motivo è lo stesso che vale nelle classi: **un metodo di istanza ha bisogno di un'istanza (un oggetto) per essere chiamato**. Da un metodo statico non c'è nessuna istanza disponibile. Serve un oggetto su cui invocarlo.

Al contrario, un metodo di istanza (default) **può** chiamare un altro metodo non statico senza problemi, perché in quel contesto un'istanza esiste.

Regola 2 — I metodi privati DEVONO avere un corpo

Potresti pensare di dichiarare un metodo privato "vuoto" (senza corpo, solo la firma):

```
private void hit(String stroke); // ✗ERRORE
```

Non è consentito. Un metodo così sarebbe **astratto** (abstract), e **i metodi astratti non possono avere il corpo** — ma un metodo privato *deve* avere un corpo perché nessuno dall'esterno può implementarlo. Quindi un metodo privato senza corpo è una contraddizione e il compilatore protesta.

Regola 3 — Da statico si chiama solo statico

Ribadendo la logica: se ti trovi dentro un **metodo statico** e vuoi chiamare un altro metodo dell'interfaccia, **quell'altro metodo deve essere anch'esso statico** (da statico a statico). Se provi a rendere `default` un metodo che viene chiamato da un contesto statico, avrai un problema; rendendolo `static` l'errore scompare.

Regola 4 — `private` e `default` non si possono combinare

I modificatori `private` e `default` **non possono stare insieme** sullo stesso metodo, perché significano due cose opposte:

- **`default`** → il metodo è pensato per essere usato (ereditato) **dai tipi che implementano l'interfaccia**. È quindi visibile all'esterno.
- **`private`** → il metodo è **solo interno all'interfaccia**, invisibile all'esterno.

```
private default void hit() { ... } //  contraddizione: NON consentito
```

Uno dice "visibile a chi implementa", l'altro dice "visibile solo qui dentro": non possono coesistere.